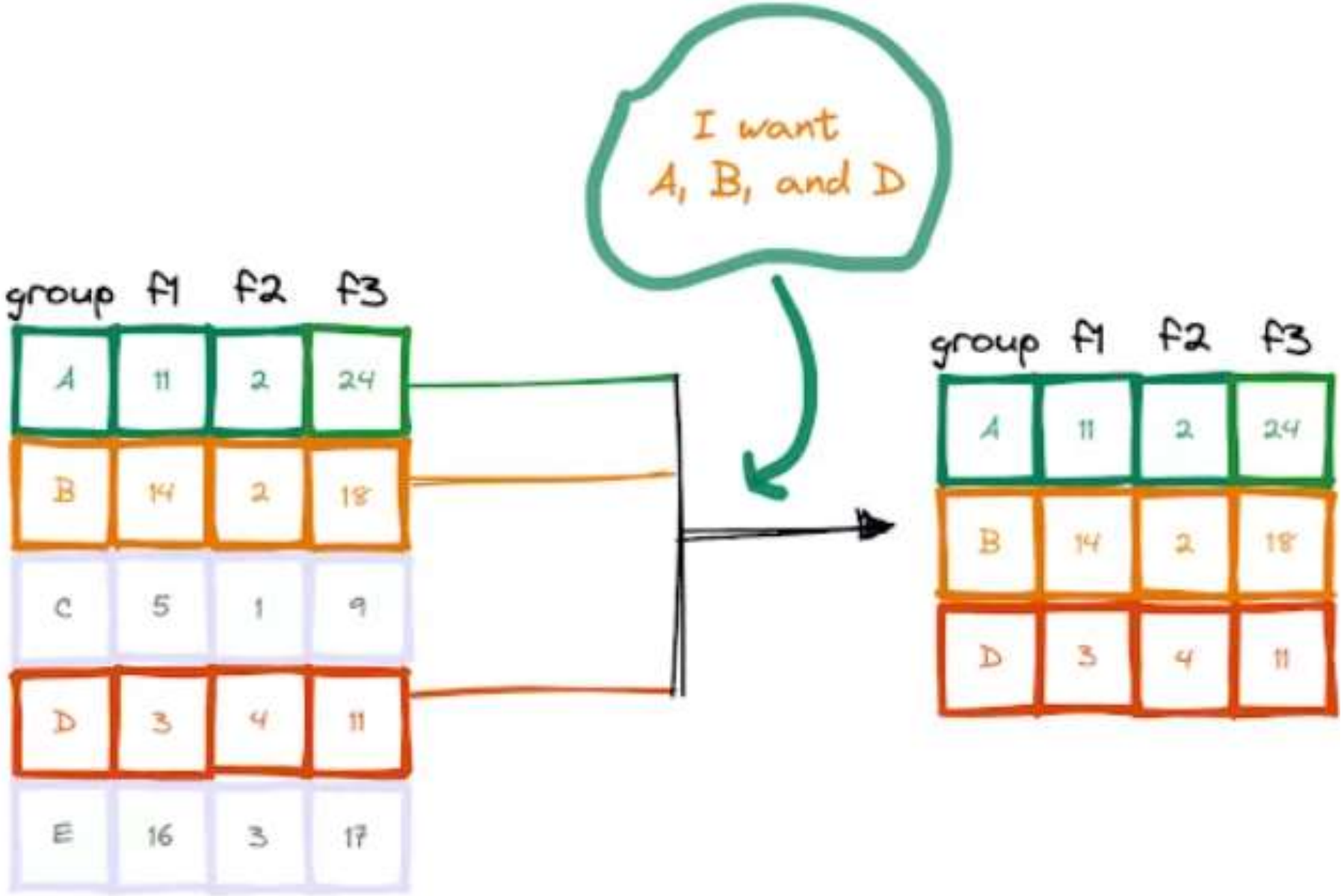
Dataselectie met pandas



Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL

Na vandaag…

Ken/weet je…

Verschillende programmeertechnieken per selectie-actie

Wanneer je welke ingebouwde pandasfunctie en –parameter moet gebruiken.

Waarom Jupyter Notebooks minder geschikt zijn om op te leveren aan klanten

HC + WC

Kan je…

Data uit DataFrames filteren, aggregeren en samenvoegen

Door DataFrames heen loopen om complexere bewerkingsacties uit te voeren

Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL

.loc[]: algemeen

.loc[] is een attribuut dat je kan opvragen om de data in je dataframes te filteren.
Let op: het is geen methode, maar een attribuut! Gebruik dus geen (), maar [].

medewerker (variabele): pd.DataFrame

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100

Welke kolommen wil ik?

```
medewerker.loc[…, …]
```

Welke rijen wil ik?

Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL

.loc[]: kolommen selecteren

De dubbele punt staat nu voor “alle rijen”.
Let goed op wanneer je bij het selecteren van kolommen een extra set vierkantshaken moet gebruiken!

medewerker (variabele): pd.DataFrame

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100

```
medewerker.loc[:, ‘naam’]
```

=>

pd.Series

```
0     Piet
1    Elske
2      Jan
3    Chris
4    Peter
5     Kees
Name: naam, dtype: object
```

```
medewerker.loc[:, [‘naam’, ‘woonplaats’]]
```

pd.DataFrame

	naam	woonplaats
0	Piet	Leiden
1	Elske	Leiden
2	Jan	Lisse
3	Chris	Berkel
4	Peter	Leiden
5	Kees	NaN

.loc[]: rijen selecteren met boolean-series

Stap 1: Definieer Boolean-serie

```
woonplaats_is_leiden = medewerker['woonplaats'] == 'Leiden'
woonplaats_is_leiden
```

✓ 0.0s

```
0     True
1     True
2    False
3    False
4     True
5    False
Name: woonplaats, dtype: bool
```

Wat zegt dit?

- De eerste twee medewerkers komen uit Leiden.
- De derde en vierde niet.
- De vijfde wel, maar de zesde weer niet.

Stap 2: Roep .loc[] aan.

```
medewerker.loc[(woonplaats_is_leiden), :]
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
4	15	Peter	Leiden	14.0	PL	140

Let goed op de ronde haakjes!

Je kan logische operatoren zó gebruiken:

- AND: (<boolean-serie>) **&** (<boolean-serie>)
- OR: (<boolean-serie>) **|** (<boolean-serie>)
- NOT: **~**(<boolean-serie>)

.loc[]: rijen selecteren op (al dan niet) lege waarden

Stap 1: Definieer Boolean-serie met na()

```
medewerker_heeft_baas = medewerker['baas'].isna()
medewerker_heeft_baas
```

✓ 0.0s

```
0    False
1    False
2    False
3     True
4    False
5    False
Name: baas, dtype: bool
```

Wat zegt en wat doet dit?

- Alleen de vierde medewerker heeft geen baas.
- Met isna() selecteer je niet-ingevulde kolommen, met notna() wel-ingevulde kolommen.

Stap 2: Roep .loc[] aan.

```
medewerker.loc[(medewerker_heeft_baas), :]
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon
3	14	Chris	Berkel	NaN	PL	130

Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL

.loc[]: rijen selecteren op delen van strings

Jelle van Barneveld

HBO-ICT

DE HAAGSE HOGESCHOOL

Definieer Boolean-serie met .str[]

```
naam_begint_met_p = medewerker['naam'].str[0] == 'P'
naam_begint_met_p
```

✓ 0.0s

```
0     True
1    False
2    False
3    False
4     True
5    False
Name: naam, dtype: bool
```

Wat zegt en wat doet dit?

- Met .str[0] wordt de eerste letter gelezen.
- De namen van de eerste en vijfde medewerker beginnen met een ‘P’.

Definieer Boolean-serie met .str.contains()

```
naam_bevat_e = medewerker['naam'].str.contains('e')
naam_bevat_e
```

✓ 0.0s

```
0     True
1     True
2    False
3    False
4     True
5     True
Name: naam, dtype: bool
```

Wat zegt en wat doet dit?

- Met .str.contains() wordt per rij bepaald of een letter in een stringkolom voorkomt
- De namen van de eerste, tweede, vijfde en zesde medewerker bevatten een ‘e’.

.loc[]: unieke waarden selecteren

Kolom selecteren

```
medewerker.loc[:, ['woonplaats']]
```

✓ 0.0s

	woonplaats
0	Leiden
1	Leiden
2	Lisse
3	Berkel
4	Leiden
5	NaN

Leiden komt maar liefst 3x voor!

Hoe halen we de dubbelen eruit?

Stap 2: Roep .loc[] aan.

```
medewerker.loc[:, ['woonplaats']].drop_duplicates('woonplaats')
```

✓ 0.0s

	woonplaats
0	Leiden
2	Lisse
3	Berkel
5	NaN

Als je meerdere kolommen in drop_duplicates wilt stoppen, moet je de kolomnamen eerst in een lijst stoppen, omringd door vierkantshaken, en die lijst vervolgens als parameter meegeven aan drop_duplicates.

Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL

De countfunctie toepassen i.c.m. group by

Count aanroepen op een DataFrame

```
medewerker.count()
✓ 0.0s

nr            6
naam          6
woonplaats    5
baas          5
functie       5
uurloon       6
dtype: int64
```

Wat zegt dit?

- Van 6 medewerkers is het nr, naam, functie en uurloon ingevuld.
- Van slechts 5 medewerkers is de woonplaats en baas ingevuld.

Count combineren met groupby

```
medewerker.groupby('woonplaats', as_index = False)['nr'].count()
✓ 0.0s
```

	woonplaats	nr
0	Berkel	1
1	Leiden	3
2	Lisse	1

Wat zegt en wat doet dit?

- Per woonplaats wordt duidelijk hoeveel medewerkers er zijn.
- De primaire-sleutelkolom moet tussen vierkantshaken staan.
- as_index = False voorkomt dat woonplaatsnamen als indexen in het dataframe worden gezien.

Andere functies toepassen i.c.m. group by

Sum combineren met groupby

```
medewerker.groupby('woonplaats', as_index = False)['uurloon'].sum()
```

✓ 0.0s

	woonplaats	uurloon
0	Berkel	130
1	Leiden	350
2	Lisse	120

Wat zegt en wat doet dit?

- Per woonplaats wordt duidelijk wat de som is van alle uurlonen.
- De kolom die je wilt aggregeren moet tussen vierkantshaken staan.
- ‘uurloon’ is in het resultaat eigenlijk geen goede kolomnaam, deze kan je beter aanpassen.

Verschillende functies gegroepeerd combineren

```
import numpy as np
medewerker.groupby('woonplaats', as_index = False)['uurloon'].agg([sum, np.mean])
```

✓ 0.0s

	woonplaats	sum	mean
0	Berkel	130	130.000000
1	Leiden	350	116.666667
2	Lisse	120	120.000000

Wat zegt en wat doet dit?

- Om van een kolom een gemiddelde aan te roepen heb je een extra library nodig: numpy.
- In agg() kun je een lijst van functies zetten, waardoor deze horizontaal gecombineerd worden. ‘Sum’ en ‘mean’ worden in dit geval in deze lijst gezet.

DataFrames horizontaal samenvoegen met merge

De tabel medewerker heeft een vreemde sleutel ‘functie’ die verwijst naar de kolom ‘code’ in de tabel ‘functie’

```
pd.merge(medewerker, functie, left_on = 'functie', how = 'inner', right_on = 'code')
```

✓ 0.0s

	nr	naam_x	woonplaats	baas	functie	uurloon	code	naam_y
0	11	Piet	Leiden	15.0	PR	100	PR	Programmeur
1	12	Elske	Leiden	15.0	PR	110	PR	Programmeur
2	13	Jan	Lisse	15.0	SA	120	SA	Systeem analist
3	14	Chris	Berkel	NaN	PL	130	PL	Projectleider
4	15	Peter	Leiden	14.0	PL	140	PL	Projectleider

- Met left_on en right_on geven we aan welke kolommen uit welke tabellen gelijk aan elkaar moeten zijn voor de horizontale samenvoeging.
- Met how geven we aan welke soort join we willen (‘inner’, ‘left’, ‘right’ of ‘outer’).
- Omdat beide tabellen een kolom ‘naam’ hebben, probeert pandas deze verschillend te maken, door achter de ene naam ‘_x’ en achter de andere naam ‘_y’ toe te voegen.

Extra hulpmiddel met merge: indicator = True

De tabel medewerker heeft een vreemde sleutel ‘functie’ die verwijst naar de kolom ‘code’ in de tabel ‘functie’

```
pd.merge(medewerker, functie, left_on = 'functie', how = 'outer', right_on = 'code', indicator = True)
```

✓ 0.0s

	nr	naam_x	woonplaats	baas	functie	uurloon	code	naam_y	_merge
0	11.0	Piet	Leiden	15.0	PR	100.0	PR	Programmeur	both
1	12.0	Elske	Leiden	15.0	PR	110.0	PR	Programmeur	both
2	13.0	Jan	Lisse	15.0	SA	120.0	SA	Systeem analist	both
3	14.0	Chris	Berkel	NaN	PL	130.0	PL	Projectleider	both
4	15.0	Peter	Leiden	14.0	PL	140.0	PL	Projectleider	both
5	16.0	Kees	NaN	15.0	NaN	100.0	NaN	NaN	left_only
6	NaN	NaN	NaN	NaN	NaN	NaN	SO	Systeem ontwerper	right_only

Door aan een outer-join ‘indicator = True’ toe te voegen zie je per rij uit de ene tabel of er een match gemaakt kan worden met de andere tabel (‘both’) en zo nee: waarom niet.

- left_only: de betreffende sleutel komt alleen voor in de linkertabel.
- right_only: de betreffende sleutel komt alleen voor de in rechtertabel.

Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL

DataFrames verticaal samenvoegen met concat

```
pd.concat([medewerker1, medewerker2], axis = 0)
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100

Let op: alle tabellen in de lijst moeten hiervoor exact dezelfde aantallen kolommen en kolomnamen hebben

Door DataFrames heen loopen voor complexer bewerkingen

```python
for index, row in medewerker.iterrows():
    huidig_uurloon = medewerker.at[index, 'uurloon']

    if(huidig_uurloon < 130):
        medewerker.at[index, 'soort_uurloon'] = 'laag'
    else:
        medewerker.at[index, 'soort_uurloon'] = 'hoog'

medewerker
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon	soort_uurloon
0	11	Piet	Leiden	15.0	PR	100	laag
1	12	Elske	Leiden	15.0	PR	110	laag
2	13	Jan	Lisse	15.0	SA	120	laag
3	14	Chris	Berkel	NaN	PL	130	hoog
4	15	Peter	Leiden	14.0	PL	140	hoog
5	16	Kees	NaN	15.0	NaN	100	laag

iterrows() zorgt ervoor dat elke rij wordt gesplitst in een indexnummer en een row, waarin alle rijwaarden opgeslagen liggen.

Met .at[] wordt het volgende gedaan:

- Per rij wordt het uurloon ingelezen.
- Een nieuwe kolom wordt aangemaakt, door een nog niet bestaande kolomnaam in te voeren, en ingevuld.

Voor- en nadelen van…

Jupyter Notebook

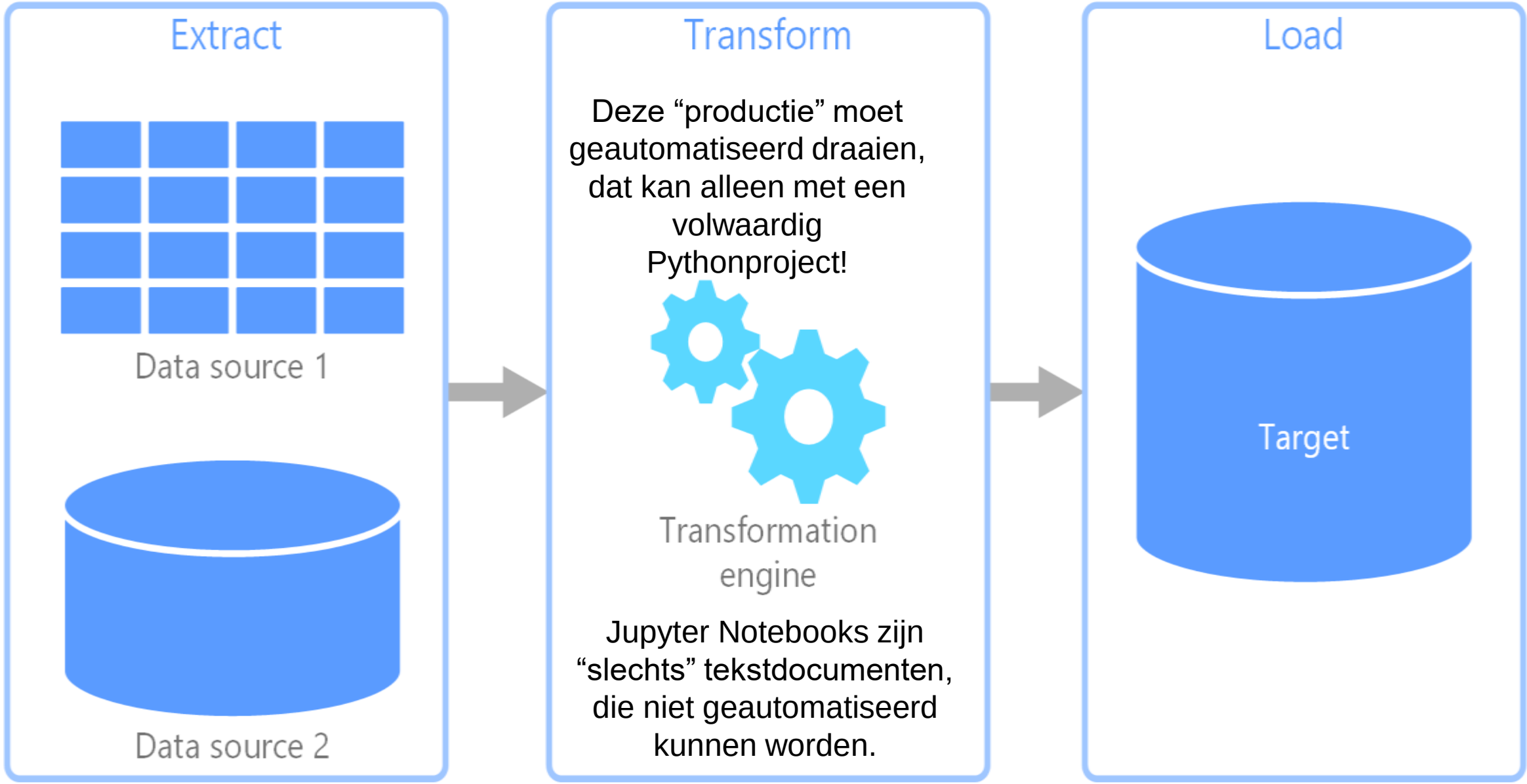
Om mijn notebook te kunnen doorlopen moet je VSCode installeren, de Jupyter-extensie installeren, de virtual environment opzetten…

- De huidige status van variabelen makkelijk in kunnen zien
- Kleine stukken Pythoncode makkelijk kunnen testen en debuggen
- Lastig uit te voeren vanaf command lines en terminals
- De opdrachtgever kan er bijna niks mee.

Pythonprojecten

- De huidige status van variabelen is lastiger in te zien (prints en breakpoints nodig)
- Kleine stukken Pythoncode kunnen lastiger getest en gedebugd worden
- Makkelijker uit te voeren vanaf command lines en terminals
- Beter over te dragen aan de opdrachtgever

Automatisering is Pythonproject-only



Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL

Van Jupyter Notebook naar Pythonproject: demo

Jelle van Barneveld

HBO-ICT

DE HAAGSE HOGESCHOOL

Na vandaag…

Ken/weet je…

Verschillende programmeertechnieken per selectie-actie

Wanneer je welke ingebouwde pandasfunctie en –parameter moet gebruiken.

Waarom Jupyter Notebooks minder geschikt zijn om op te leveren aan klanten

HC + WC

Kan je…

Data uit DataFrames filteren, aggregeren en samenvoegen

Door DataFrames heen loopen om complexere bewerkingsacties uit te voeren

Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL